MOVIE-APP · CLIENT/SRC/LIB/PAGINATION.TS

Как работает `getPageRange()`

Объяснение «на пальцах»: что такое **range()**, **siblings**, **totalSlots**, откуда берутся многоточия и почему ряд кнопок всегда одной ширины.

Одна фраза

Функция получает **два числа** (на какой мы странице и сколько страниц всего) и возвращает **список того, что нарисовать** в ряду пагинации: номера страниц и многоточия.

`currentPage = 10`

`totalPages = 20`

`siblings = 1`

→ getPageRange →

1 … 9 10 11 … 20

```
[1, 'ellipsis-left', 9, 10, 11, 'ellipsis-right', 20]
```

Легенда цветов (используется во всём документе)

- **10** — текущая страница
- **9** — сосед (sibling)
- **1** — первая / последняя — видны всегда
- **…** — многоточие: «тут спрятаны страницы»

Три правила, которые реализует код

1. **Первая и последняя** страницы видны всегда, чтобы прыгнуть в начало или конец одним кликом.
2. Вокруг **текущей** видно по **siblings** соседей с каждой стороны.
3. Если между крайней страницей и соседями что-то **пропущено**, на этом месте стоит **многоточие**.

1 range(from, to): «числа от … до … включительно»

```
function range(from: number, to: number): number[] {
  return Array.from({ length: to - from + 1 }, (_, i) => from + i)
}
```

Это всего лишь способ написать список чисел подряд. `range(3, 7)` → `[3, 4, 5, 6, 7]`.

Как это получается по шагам

`Array.from({ length: N }, fn)` создаёт массив из N элементов и для каждого вызывает `fn(_, i)`, где `i` — это индекс: 0, 1, 2, … Мы к каждому индексу прибавляем `from`.

индекс i	0	1	2	3	4
from + i	3 + 0	3 + 1	3 + 2	3 + 3	3 + 4
результат	3	4	5	6	7

Почему length = to − from + 1?

Разница 7 − 3 = 4, но чисел от 3 до 7 — пять. Это «эффект столбов и пролётов»: между пятью столбами четыре пролёта. Поэтому к разнице всегда прибавляем единицу.



5 столбов (чисел), 4 пролёта (разница). length = 7 − 3 + 1 = 5 ✓

Где range() используется внутри getPageRange

- `range(1, totalPages)` — показать вообще все страницы, когда их мало.
- `range(1, leftCount)` — «начало»: первые 5 страниц подряд.
- `range(totalPages - rightCount + 1, totalPages)` — «конец»: последние 5 страниц подряд.
- `range(leftSibling, rightSibling)` — «середина»: сосед, текущая, сосед.

2 siblings: сколько соседей показывать

Слово *sibling* переводится как «брат/сестра», в UI так называют соседние элементы. Здесь **siblings = сколько номеров видно слева и справа от текущей страницы**.

Одна и та же текущая страница 10 из 20 при разных siblings

siblings = 0	1 … **10** … 20	вокруг 10 никого
siblings = 1	1 … 9 **10** 11 … 20	по одному соседу: 9 и 11
siblings = 2	1 … 8 9 **10** 11 12 … 20	по два соседа: 8 9 и 11 12
siblings = 3	1 … 7 8 9 **10** 11 12 13 … 30	по три соседа (всего 30 страниц)

В проекте используется siblings = 1 — самый компактный вариант, который всё ещё показывает «куда назад» и «куда вперёд».

leftSibling и rightSibling — границы «окна» вокруг текущей

```
const leftSibling  = Math.max(currentPage - siblings, 1)
const rightSibling = Math.min(currentPage + siblings, totalPages)
```

Обычно это просто `currentPage - 1` и `currentPage + 1`. `Math.max / Math.min` нужны только для краёв: чтобы не получить страницу 0 или 21.

currentPage = 10

7 8 9 **10** 11 12 13

leftSibling = max(9, 1) = **9** · rightSibling = min(11, 20) = **11**

currentPage = 1 (левый край)

~~0~~ **1** 2 3 4

1 − 1 = 0, но страницы 0 нет → `Math.max(0, 1)` = **1**. Левый сосед «упёрся» в первую страницу.

currentPage = 20 (правый край)

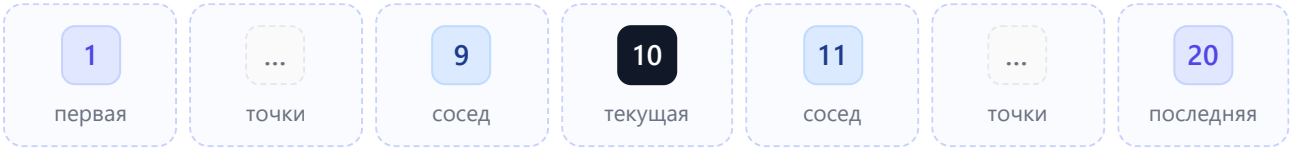
17 18 19 **20** ~~21~~

20 + 1 = 21, но страниц всего 20 → `Math.min(21, 20)` = **20**.

3 totalSlots: сколько «мест» в ряду

```
const totalSlots = siblings * 2 + 5
```

Ряд пагинации — это как полка с фиксированным числом ячеек. Считаем, сколько ячеек нужно в самом «полном» случае (середина списка):



5 постоянных — первая + последняя + текущая + 2 многоточия

\+ **siblings × 2** — соседи слева и справа

= **7** при siblings = 1

Зачем знать totalSlots? Для первой проверки

```
if (totalPages <= totalSlots) {
  return range(1, totalPages)   // страниц мало — показываем все, без точек
}
```

Если страниц меньше или ровно столько, сколько ячеек, прятать нечего. Многоточие вместо реального номера только мешало бы.

totalPages = 3	1 **2** 3	3 ≤ 7 → все
totalPages = 7	1 2 3 **4** 5 6 7	7 ≤ 7 → все, ровно заполнили полку
totalPages = 8	1 ... 3 **4** 5 ... 8	8 > 7 → не влезает, появляются точки

Главная выгода фиксированной ширины

При переходе между страницами ряд **не меняет длину**, кнопки не «уезжают» из-под курсора. Сравните: в старом варианте окно из 5 кнопок на разных страницах давало то 3, то 4 кнопки.

4 showLeftDots / showRightDots: нужно ли многоточие

```
const showLeftDots  = leftSibling > 2
const showRightDots = rightSibling < totalPages - 1
```

Многоточие ставят только тогда, когда между крайней страницей и ближайшим соседом есть **хотя бы одна спрятанная** страница. Вопрос, который задаёт код: *«между 1 и leftSibling что-то пропущено?»*

Левая сторона: сравниваем leftSibling с числом 2

leftSibling = 2 (currentPage = 3)	1 2 **3**	между 1 и 2 ничего нет → **2 > 2 ложь** → точки **не нужны**
leftSibling = 3 (currentPage = 4)	1 2 3 **4**	между 1 и 3 спрятана **2** → **3 > 2 истина** → точки **нужны**
leftSibling = 9 (currentPage = 10)	1 … 9 **10**	спрятаны 2…8 → точки нужны

Почему именно `> 2`, а не `> 1`: если leftSibling = 2, то слева от него только «1», которая и так видна. Прятать нечего.

Правая сторона — зеркально: сравниваем с totalPages − 1

rightSibling = 19 (currentPage = 18, total 20)	**18** 19 20	между 19 и 20 ничего → **19 < 19 ложь** → без точек
rightSibling = 18 (currentPage = 17)	**17** 18 19 20	спрятана **19** → **18 < 19 истина** → точки

Два флага → четыре комбинации → три ветки кода

showLeftDots	showRightDots	где мы	ветка
нет	да	в начале	1-й `if`
да	нет	в конце	2-й `if`
да	да	в середине	последний `return`
нет	нет	невозможно: это значило бы, что всё влезло, а такой случай уже отсечён проверкой totalPages ≤ totalSlots	

5 Три ветки: начало, конец, середина

Ветка «начало»: точки только справа

```
if (!showLeftDots && showRightDots) {
  const leftCount = 3 + 2 * siblings          // = 5
  return [...range(1, leftCount), 'ellipsis-right', totalPages]
}
```

1 **2** 3 4 5 … 20 currentPage = 2 → `[1, 2, 3, 4, 5, 'ellipsis-right', 20]`

Откуда **3 + 2·siblings = 5**? Хотим занять все 7 ячеек: 5 номеров подряд + точки + последняя = 7. Иными словами, leftCount = totalSlots − 2 (минус точки и минус последняя страница).

Ветка «конец»: точки только слева

```
if (showLeftDots && !showRightDots) {
  const rightCount = 3 + 2 * siblings         // = 5
  return [1, 'ellipsis-left', ...range(totalPages - rightCount + 1, totalPages)]
}
```

1 … 16 17 18 **19** 20

currentPage = 19 → `[1, 'ellipsis-left', 16, 17, 18, 19, 20]`

`range(20 - 5 + 1, 20)` = `range(16, 20)` — последние пять страниц. Снова «+1» из-за столбов и пролётов.

Ветка «середина»: точки с обеих сторон

```
return [
  1, 'ellipsis-left',
  ...range(leftSibling, rightSibling),
  'ellipsis-right', totalPages,
]
```

1 … 9 **10** 11 … 20

currentPage = 10 → `[1, 'ellipsis-left', 9, 10, 11, 'ellipsis-right', 20]`

Здесь всё буквально: первая, точки, окно «сосед–текущая–сосед», точки, последняя.

Зачем точки называются по-разному: 'ellipsis-left' и 'ellipsis-right'

Это не для вида. В React каждому элементу списка нужен уникальный `key`. Числа уникальны сами по себе, а если бы оба многоточия были строкой `'ellipsis'`, React выдал бы предупреждение о дублирующихся ключах.

6 Как ряд «едет» при клике с 1 по 20

Все 20 состояний для totalPages = 20, siblings = 1. Заметьте: ширина всегда 7 ячеек, первая и последняя стоят на месте, а окно вокруг текущей сдвигается на одну позицию за клик.

стр.	что нарисуется	ветка
1	**1** 2 3 4 5 … 20	начало
2	1 **2** 3 4 5 … 20	начало
3	1 2 **3** 4 5 … 20	начало
4	1 … 3 **4** 5 … 20	середина
5	1 … 4 **5** 6 … 20	середина
6	1 … 5 **6** 7 … 20	середина
7	1 … 6 **7** 8 … 20	середина
8	1 … 7 **8** 9 … 20	середина
9	1 … 8 **9** 10 … 20	середина
10	1 … 9 **10** 11 … 20	середина
11	1 … 10 **11** 12 … 20	середина
12	1 … 11 **12** 13 … 20	середина
13	1 … 12 **13** 14 … 20	середина
14	1 … 13 **14** 15 … 20	середина
15	1 … 14 **15** 16 … 20	середина
16	1 … 15 **16** 17 … 20	середина
17	1 … 16 **17** 18 … 20	середина
18	1 … 16 17 **18** 19 20	конец
19	1 … 16 17 18 **19** 20	конец
20	1 … 16 17 18 19 **20**	конец

7 Трассировка: как функция «думает» на конкретных числах

siblings = 1, totalSlots = 7. Читайте строку слева направо — это ровно порядок строк в коде.

вход	leftSibling	rightSibling	точки слева?	точки справа?	ветка	результат
1 из 20	1	2	нет	да	начало	**1** 2 3 4 5 … 20
3 из 20	2	4	нет	да	начало	1 2 **3** 4 5 … 20
4 из 20	3	5	да	да	середина	1 … 3 **4** 5 … 20
10 из 20	9	11	да	да	середина	1 … 9 **10** 11 … 20
17 из 20	16	18	да	да	середина	1 … 16 **17** 18 … 20
18 из 20	17	19	да	нет	конец	1 … 16 17 **18** 19 20
20 из 20	19	20	да	нет	конец	1 … 16 17 18 19 **20**
3 из 5	—	—	—	—	все страницы	1 2 **3** 4 5
4 из 7	—	—	—	—	все страницы	1 2 3 **4** 5 6 7

Как проверить самому

Откройте консоль браузера или Node и вызовите функцию с разными числами. Если результат совпадает с таблицей, вы поняли алгоритм.

```
getPageRange(1, 20, 1)   // [1, 2, 3, 4, 5, 'ellipsis-right', 20]
getPageRange(4, 20, 1)   // [1, 'ellipsis-left', 3, 4, 5, 'ellipsis-right', 20]
getPageRange(20, 20, 1)  // [1, 'ellipsis-left', 16, 17, 18, 19, 20]
getPageRange(3, 5, 1)    // [1, 2, 3, 4, 5]
```

8 Весь код с пометками

```
export type PageItem = number | 'ellipsis-left' | 'ellipsis-right'
```

Один элемент ряда: либо номер, либо одно из двух многоточий.

```
function range(from, to) {
  return Array.from({ length: to - from + 1 }, (_, i) => from + i)
}
```

Список чисел от from до to включительно. «+1» — столбы и пролёты.

```
export function getPageRange(currentPage, totalPages, siblings) {
```

Вход: где мы, сколько всего, сколько соседей показывать.

```
  const totalSlots = siblings * 2 + 5
```

Сколько ячеек в ряду. При siblings = 1 → 7.

```
  if (totalPages <= totalSlots) return range(1, totalPages)
```

Всё влезает → показываем все страницы, выходим.

```
  const leftSibling  = Math.max(currentPage - siblings, 1)
  const rightSibling = Math.min(currentPage + siblings,
totalPages)
```

Границы окна вокруг текущей. max/min не дают выйти за 1 и totalPages.

```
  const showLeftDots  = leftSibling > 2
  const showRightDots = rightSibling < totalPages - 1
```

Есть ли спрятанные страницы между краем и окном?

```
  if (!showLeftDots && showRightDots) {
    const leftCount = 3 + 2 * siblings
    return [...range(1, leftCount), 'ellipsis-right', totalPages]
  }
```

Начало. 5 номеров подряд, точки, последняя.

1 2 3 4 5 … 20

```
  if (showLeftDots && !showRightDots) {
    const rightCount = 3 + 2 * siblings
    return [1, 'ellipsis-left', ...range(totalPages - rightCount +
1, totalPages)]
  }
```

Конец. Первая, точки, последние 5 номеров.

1 … 16 17 18 19 20

```
  return [1, 'ellipsis-left', ...range(leftSibling, rightSibling),
'ellipsis-right', totalPages]
}
```

Середина. Первая, точки, окно, точки, последняя.

1 … 9 10 11 … 20

Что с этим делать дальше

Функция чистая: без React, без состояния. Компонент пагинации просто делает `getPageRange(currentPage, totalPages, 1).map(...)` и для числа рисует кнопку, для строки — многоточие. Вся «умная» часть живёт здесь, а разметка остаётся тупой и простой.